

Tema 3: Capa de Servicios REST con Spring

La capa REST actúa como frontera de nuestra aplicación, recibiendo las peticiones HTTP del frontend (React), transformando los datos (JSON) a objetos Java (DTOs), y delegando la ejecución a la capa de servicios (Lógica de Negocio).

1. Desarrollo de Controladores

Cada servicio local suele tener su contrapartida en la capa REST mediante una clase controladora anotada con `@RestController`. Estas clases utilizan inyección de dependencias (`@Autowired`) para acceder a la capa de servicios.

1.1. Mapeo de Peticiones y Parámetros

Para definir qué método atiende cada petición HTTP, empleamos anotaciones de mapeo como `@GetMapping`, `@PostMapping`, `@PutMapping`, etc.

La extracción de datos de la petición HTTP se realiza mediante tres anotaciones fundamentales:

- `@PathVariable`: Extrae valores directamente de la URI (ej. `/catalog/movies/{id}`).
 - `@RequestParam`: Extrae parámetros de la cadena de consulta (*query string*, ej. `?date=2026-05-24`).
 - `@RequestBody`: Convierte el cuerpo de la petición (JSON) a un DTO.
 - **Regla de Examen:** Olvidar la anotación `@RequestBody` al recibir un objeto complejo en un método `@PostMapping` es un error crítico frecuentemente evaluado.
-

2. Validación Básica de Datos

Es una mala práctica sobrecargar la lógica de negocio con validaciones de formato (ej. comprobar que un correo es válido o que un texto no supera 60 caracteres). Delegamos esto a la capa REST utilizando la API **Bean Validation**.

En las clases DTO, empleamos anotaciones como `@NotNull`, `@Size`, `@Min`, o `@Max`.

```
public class PurchaseParamsDto {  
    // ...  
}
```

```
@NotNull
@Size(min=16, max=16)
public String getCreditCard() { return creditCard; }
// ...
}
```

Para que Spring ejecute estas reglas al recibir una petición, debemos anteceder el parámetro del controlador con la anotación `@Validated`:

```
public PurchaseDto buy(@RequestAttribute Long userId, @Validated
@RequestBody PurchaseParamsDto params)
```

Si la validación falla, Spring lanza automáticamente una `MethodArgumentNotValidException` y aborta la petición devolviendo un error HTTP **400 (Bad Request)**.

3. Gestión de Excepciones

Nunca debemos llenar los controladores de bloques `try-catch`. Spring nos proporciona mecanismos elegantes para interceptar excepciones y traducirlas a códigos de estado HTTP.

- **Global (`@ControllerAdvice`)**: Las excepciones comunes (como `InstanceNotFoundException` o fallos de validación) se centralizan en una clase global, en nuestro caso `CommonControllerAdvice`.
 - **Anotaciones clave**: Utilizamos `@ExceptionHandler` para indicar qué excepción capturar, y `@ResponseStatus` para asignar el código HTTP de respuesta (ej. `HttpStatus.NOT_FOUND` para un elemento no encontrado).
 - **Retorno**: El método debe devolver un objeto `ErrorsDto` que Spring transformará automáticamente a JSON con el mensaje de error estructurado para el frontend.
-

4. Internacionalización de Mensajes (i18n)

Los mensajes de error devueltos al frontend deben estar en el idioma del usuario. Spring resuelve esto inyectando un objeto `Locale`, cuyo valor se deduce de la cabecera HTTP `Accept-Language` enviada por el navegador.

Los textos se almacenan de forma externa en ficheros `.properties` (`messages_es.properties`, `messages_en.properties`, etc.) bajo el esquema `clave=valor`. Para recuperar el texto traducido, inyectamos el bean `MessageSource` y ejecutamos `messageSource.getMessage(...)` proporcionando la clave y el `Locale`.

5. Autenticación, Autorización y Control de Acceso

5.1. JSON Web Tokens (JWT)

Nuestro sistema es *stateless* (sin estado), lo que significa que el servidor no guarda sesiones en memoria. Utilizamos tokens JWT, los cuales encapsulan la identidad del usuario y sus roles de forma cifrada (firmados mediante el algoritmo HMAC-SHA). Al hacer login, el servidor genera este token y se lo entrega al cliente, quien deberá adjuntarlo en la cabecera

`Authorization: Bearer <token>` en las peticiones sucesivas.

5.2. El Filtro de Seguridad (`JwtFilter`)

Antes de que una petición llegue al controlador, atraviesa el `JwtFilter`. Este componente intercepta la petición, verifica la validez y caducidad de la firma del token, extrae el identificador del usuario (`userId`) y su rol, y configura el contexto de seguridad de Spring (`SecurityContext`).

5.3. Autorización (`SecurityConfig`)

La clase `SecurityConfig` define de manera declarativa quién puede acceder a qué rutas.

```
// Ejemplo de configuración
.requestMatchers(HttpMethod.POST,
"/shopping/purchases/*/deliver").hasRole("TICKETSELLER")
.requestMatchers(HttpMethod.POST, "/shopping/buy").hasRole("USER")
```